Pixeon Image Viewer

Medical Edition

Relatório Técnico

Desafio Técnico — Analista Desenvolvedor C++ Júnior

Código fonte: neuxxkk/pixeon-image-viewer

Candidato: Vitor Neuenschwander Mendes da Silva

Contato: vitornms@gmail.com

LinkedIn: linkedin.com/in/vitornms

GitHub: github.com/neuxxkk

Data: 2 de abril de 2026

Tecnologias: C++17 • Qt 6 • CMake • QTest

Sumário

1 Introdução **2**

2 Metodologia e Arquitetura **2**

2.1 Padrão MVC 2

2.2 Sistema de Build — CMake 2

2.3 Linguagem — C++17 3

2.4 Padronização e Nomenclatura 3

3 Formatos de Imagem Suportados **4**

4 Técnicas Aplicadas **4**

4.1 Lookup Table (LUT) para Brilho e Contraste 4

4.2 Renderização de Alta Precisão (Pixel-Perfect) 5

4.3 Integração de Controles e MenuUI 5

4.4 Ferramenta Caliper (Régua Médica) 6

4.5 Inversão de Polaridade (Negativo) 6

5 Funcionalidades Implementadas **7**

5.1 Requisitos Obrigatórios 7

5.2 Funcionalidades Bônus 7

6 Testes Unitários **7**

7 Dificuldades Encontradas **8**

7.1 LUT com Canal Separado e Overflow 8

7.2 Gestão de Coordenadas Multi-Transformação 8

7.3 Interpolação Adaptativa no Zoom 9

8 Possíveis Melhorias Futuras **9**

9 Avaliação do Resultado **9**

1. Introdução

Este documento descreve as decisões técnicas, metodologia e resultados obtidos no desenvolvimento do **Pixeon Image Viewer**, aplicação desktop de visualização e manipulação de imagens médicas desenvolvida como desafio técnico para a vaga de Analista Desenvolvedor C++ Júnior na Pixeon.

O projeto evoluiu intencionalmente além dos requisitos mínimos do enunciado, incorporando funcionalidades de apoio ao diagnóstico médico, como calibração de medições em milímetros e inversão de polaridade radiográfica, refletindo o contexto do produto da empresa.

Repositório e Executável

Código-fonte: `https://github.com/neuxxkk/pixeon-image-viewer`
O repositório contém o código-fonte completo, o arquivo de projeto CMake e o executável Windows empacotado com `windeployqt`.

2. Metodologia e Arquitetura

2.1. Padrão MVC

O projeto adota o padrão **Model-View-Controller (MVC)** como espinha dorsal da arquitetura, separando claramente as três responsabilidades:

2.2. Sistema de Build — CMake

Optou-se por **CMake** em detrimento do `qmake` por três razões objetivas:

1. **Suporte a IDEs modernas:** integração nativa com VSCode (*CMake Tools*), CLion e Qt Creator 6+.
2. **Separação de targets:** o projeto principal e os testes unitários são targets independentes, compilados e executados separadamente via `ctest`.
3. **Adoção da indústria:** CMake é o padrão de facto em projetos C++ corporativos modernos.

Tabela 1: Distribuição de responsabilidades no padrão MVC

Camada	Responsabilidade
Model	
`ImageModel`, `ImageCollection`	Encapsula o estado da imagem (original e processada), parâmetros de ajuste (brilho, contraste, rotação, inversão) e a coleção de imagens abertas simultaneamente.
View	
`MainWindow`, `ImageViewWidget`	Gerencia a interface gráfica Qt, renderiza a imagem com suporte a zoom, pan e anotações, e exibe a lista lateral de imagens abertas.
Controller	
`ImageProcessor`	Contém algoritmos puros de processamento de imagem, completamente isolados da camada de interface. Facilita testes unitários independentes.

2.3. Linguagem — C++17

O padrão C++17 foi escolhido por oferecer ferramentas que tornam o código mais seguro e expressivo sem custo de runtime:

- `std::optional<QImage>` para representar estado "sem imagem carregada" de forma type-safe, eliminando sentinelas frágeis como `QImage()` vazio ou flags booleanas separadas.
- `std::clamp` para garantir limites de valores sem risco de overflow silencioso.
- `std::array<uint8_t, 256>` alocado em stack para a LUT, evitando alocação dinâmica em hot paths.
- `if constexpr`, structured bindings e deduções de tipo (`auto`) onde aplicável.

2.4. Padronização e Nomenclatura

Embora o presente relatório e a interface final do usuário utilizem o idioma português para facilitar a comunicação, optou-se pela **nomenclatura em inglês** para todos os elementos técnicos e de desenvolvimento do projeto, incluindo:

- Nomes de arquivos e diretórios da estrutura de código;
- Nomes de classes, funções, variáveis e membros;
- Mensagens de *commit* no histórico do Git.

Esta decisão alinha o projeto às melhores práticas da indústria global de *software*, garantindo consistência com as bibliotecas padrão (STL), o *framework* Qt e facilitando a futura manutenção por equipes multiculturais ou o uso de ferramentas internacionais de análise de código.

3. Formatos de Imagem Suportados

O enunciado solicitava suporte a pelo menos um dos formatos BMP, PNG ou JPEG (*bônus* para mais de um). O projeto suporta **os três formatos**, sem dependência de bibliotecas externas, pois o Qt 6 os gerencia nativamente via `QImage::load()`.

Tabela 2: Formatos de imagem suportados

Formato	Extensões	Característica relevante
BMP	`.bmp`	Sem compressão, leitura trivial
PNG	`.png`	Canal alfa; convertido para RGB32
JPEG	`.jpg`, `.jpeg`	Compressão com perda; mais comum em PACS

Todas as imagens são convertidas imediatamente para o formato interno `QImage::Format_RGB32` no momento do carregamento. Isso garante que os algoritmos de processamento operem sempre sobre um layout de memória uniforme (4 bytes por pixel: R, G, B + padding), independente do formato original.

4. Técnicas Aplicadas

4.1. Lookup Table (LUT) para Brilho e Contraste

A técnica de **Lookup Table** é o método profissional para aplicar transformações tonais em imagens. A ideia central é:

Princípio da LUT

Em vez de calcular a fórmula de brilho/contraste para cada um dos potencialmente milhões de pixels, calcula-se apenas **256 vezes** (uma por possível valor de canal 0–255) e armazena-se o resultado em uma tabela. Cada pixel é então transformado por uma simples consulta de índice: $O(1)$ por pixel.

A fórmula aplicada combina contraste e brilho em uma única passagem:

$$\text{LUT}[i] = \text{clamp}(f_c \cdot (i - 128) + 128 + B,\ 0,\ 255) \tag{1}$$

onde $f_c = \frac{C+100}{100}$ é o fator de contraste (intervalo $[0, 2]$) e $B \in [-100, 100]$ é o deslocamento de brilho. O `std::clamp` garante que o resultado nunca ultrapasse os limites do tipo `uint8_t`, evitando overflow silencioso.

O acesso à memória da imagem é feito diretamente via `scanLine(y)`, evitando a sobrecarga de chamadas ao método `pixel(x, y)`:

```
for (int y = 0; y < h; ++y) {
    QRgb* line = reinterpret_cast<QRgb*>(m_processed.scanLine(y));
    for (int x = 0; x < w; ++x) {
        const QRgb px = line[x];
        line[x] = qRgb(lut[qRed(px)],
                       lut[qGreen(px)],
                       lut[qBlue(px)]);
    }
}
```

Listing 1: Aplicação da LUT com acesso direto à memória

4.2. Renderização de Alta Precisão (Pixel-Perfect)

Para apoiar o diagnóstico médico detalhado, o widget de visualização implementa uma lógica de renderização adaptativa baseada no nível de ampliação:

- **Zoom < 10×:** utiliza interpolação bilinear (`Qt::SmoothTransformation`) para suavizar transições e evitar aliasing.
- **Zoom > 10×:** a suavização é desativada (`nearest-neighbor`), permitindo que o médico visualize os valores reais dos pixels sem interpolação artificial que poderia mascarar artefatos.
- **Zoom > 15×:** ativa-se uma **grade de contorno de pixels**. Utilizando uma `QPen` cosmética (largura fixa de 1px independente da escala), o sistema desenha bordas translúcidas ao redor de cada pixel, facilitando a contagem e análise de estruturas em nível microscópico.

4.3. Integração de Controles e MenuUI

A interface foi reformulada para oferecer flexibilidade total ao usuário, inspirada em estações de trabalho radiológicas (*Workstations*) profissionais:

- **Agrupamento Funcional:** Os controles na barra lateral são organizados em `QGroupBox` (Ajustes Tonais, Zoom, Rotação).
- **Visibilidade Dinâmica:** O menu "Ferramentas" permite que o usuário ligue/desligue a exibição de cada grupo individualmente, limpando a interface de elementos desnecessários para o fluxo atual.
- **Persistência de Visão:** Adição do menu "Exibir", que permite reabrir a barra lateral de ajustes caso ela seja fechada acidentalmente, garantindo que o usuário nunca perca o acesso aos controles.

4.4. Ferramenta Caliper (Régua Médica)

A régua de medição (*caliper*) é a funcionalidade de maior complexidade geométrica do projeto. Ela exige o mapeamento bidirecional entre dois espaços de coordenadas:

- **Coordenadas do widget:** onde o usuário clica com o mouse.
- **Coordenadas lógicas da imagem:** posição real no pixel original, independente de zoom, pan ou rotação aplicados.

A conversão inversa requer desfazer as transformações acumuladas pelo `QPainter` (translação de pan, escala de zoom, rotação), o que foi resolvido através da **inversa da matriz de transformação** composta:

$$\mathbf{p}_{\text{imagem}} = \mathbf{M}^{-1} \cdot \mathbf{p}_{\text{widget}} \tag{2}$$

onde $\mathbf{M} = T_{\text{pan}} \cdot S_{\text{zoom}} \cdot R_{\theta}$.

A medição é exibida em **milímetros**, utilizando uma calibração simulada de $0{,}1$ mm/pixel — valor representativo de modalidades de imagem médica como radiografia digital. Em um sistema PACS real, esse valor seria lido diretamente dos metadados DICOM (*Pixel Spacing*).

4.5. Inversão de Polaridade (Negativo)

O filtro de inversão (valor$' = 255 -$ valor) simula a visualização de filmes radiográficos tradicionais, onde ossos aparecem escuros sobre fundo claro. É implementado via `QImage::invertPixels()` do Qt, operação otimizada internamente pela biblioteca.

5. Funcionalidades Implementadas

5.1. Requisitos Obrigatórios

- Abertura de imagens via `QFileDialog` com filtro por extensão.
- Suporte a BMP, PNG e JPEG.
- Exibição centralizada da imagem com fundo preto.
- Controle de brilho via `QSlider` (intervalo $[-100, 100]$).
- Controle de contraste via `QSlider` (intervalo $[-100, 100]$).
- Sliders desabilitados enquanto nenhuma imagem estiver aberta.

5.2. Funcionalidades Bônus

- **Bônus 4c — Drag do mouse:** arrastar horizontalmente altera o brilho; verticalmente, o contraste. Os sliders são atualizados em tempo real.
- **Bônus 5 — Múltiplas imagens:** lista lateral com miniaturas; seleção troca a imagem exibida sem fechar as demais.
- **Bônus 6a — Zoom:** scroll do mouse com interpolação bilinear e centralizando imagem com pan.
- **Bônus 6b — Pan:** arrasto com botão direito; imagem retorna ao centro com duplo clique.
- **Bônus 6c — Rotação:** controle de rotação em 90° e ângulo livre por slider.
- **Bônus 7 — Surpresa:** Ferramenta Caliper (régua calibrada em mm) e inversão de polaridade radiográfica.

6. Testes Unitários

Os testes foram implementados com o framework **Qt Test (QTest)**, incluído nativamente no Qt 6, dispensando dependências externas. Cada caso de teste é um método de slot privado da classe de teste:

Os testes são executados via `ctest` a partir do diretório de build:

Tabela 3: Casos de teste implementados

Caso de teste	O que verifica
`testNeutralAdjustment`	Brilho $= 0$, contraste $= 0$ retorna imagem bitwise-idêntica à original.
`testBrightnessClamp`	Brilho $= +100$ em pixel $(250, 250, 250)$ resulta em $(255, 255, 255)$; sem overflow.
`testNegativeBrightness`	Brilho $= -100$ em pixel $(10, 10, 10)$ resulta em $(0, 0, 0)$; sem underflow.
`testContrastMidpoint`	Pixel exato em 128 permanece 128 para qualquer fator de contraste.
`testLoadInvalidFile`	`loadFromFile()` retorna `false` e `isLoaded()` permanece `false` com caminho inválido.
`testLoadValidFile`	`loadFromFile()` retorna `true` e imagem tem dimensões corretas com arquivo PNG válido.

```
cd build && ctest --output-on-failure
```

Listing 2: Execução dos testes

7. Dificuldades Encontradas

7.1. LUT com Canal Separado e Overflow

A principal dificuldade técnica foi garantir que a LUT tratasse cada canal RGB de forma independente, sem que a transformação de contraste — centralizada em 128 e de natureza não-linear — interagisse incorretamente com o deslocamento de brilho. O risco concreto era um **overflow silencioso** ao fazer `static_cast<uint8_t>` em valores fora do intervalo $[0, 255]$, que em C++ é comportamento indefinido para inteiros com sinal. A solução foi aplicar `std::clamp` sobre o tipo `double` antes da conversão, garantindo sempre um valor seguro.

7.2. Gestão de Coordenadas Multi-Transformação

Converter a posição do clique do mouse (espaço do widget) para a coordenada real na imagem original, após zoom, pan e rotação acumulados, exigiu a **inversão da matriz de transformação composta** do `QPainter`. O Qt disponibiliza `QTransform::inverted()`, mas a construção manual da composição correta de transformações (ordem importa: rotação antes de escala antes de translação) demandou atenção especial para a ferramenta caliper funcionar com precisão.

7.3. Interpolação Adaptativa no Zoom

Inicialmente, a imagem era sempre escalada com `SmoothTransformation`, o que borrava os pixels ao ampliar. A solução foi alternar para `FastTransformation` (nearest-neighbor) quando o zoom ultrapassa $1\times$, e adicionar a grade de pixels a partir de $8\times$, replicando o comportamento de editores profissionais como o GIMP.

8. Possíveis Melhorias Futuras

1. **Processamento em GPU via OpenGL.** Migrar os filtros de brilho/contraste para *shaders* GLSL utilizando `QOpenGLWidget`, eliminando a latência CPU para imagens de alta resolução (ex.: radiografias 4096×4096).
2. **Suporte nativo a DICOM via DCMTK.** Integrar a biblioteca DCMTK para leitura de metadados reais de calibração (*Pixel Spacing*), eliminando a calibração simulada atual e tornando as medições do caliper clinicamente válidas.
3. **Janelamento (*Windowing*).** Implementar controles de *Window Level* / *Window Width* para imagens de 12-bit ou 16-bit, padrão em Tomografia Computadorizada e Ressonância Magnética.
4. **Processamento assíncrono.** Mover a aplicação da LUT para uma thread separada via `QtConcurrent::run`, mantendo a interface responsiva em imagens grandes.
5. **Filtros convolucionais.** Adicionar filtros como *sharpening*, suavização gaussiana e detecção de bordas (Sobel/Canny) via `std::array` de kernels aplicados com `QImage`.

9. Avaliação do Resultado

O projeto entregou todos os requisitos obrigatórios e todos os bônus especificados no enunciado, acrescido da funcionalidade extra de caliper médico. A arquitetura MVC adotada manteve o código testável e extensível: o `ImageProcessor` não tem dependência alguma de Qt Widgets, podendo ser testado em isolamento completo.

A decisão de implementar funcionalidades orientadas ao domínio médico — calibração em mm, inversão de polaridade, interpolação adaptativa para diagnóstico — foi deliberada: demonstra compreensão do contexto de negócio da Pixeon (PACS, imagens médicas, DICOM) e da responsabilidade que esse software carrega para profissionais de saúde.

Do ponto de vista pessoal, a maior evolução foi a gestão de coordenadas multi-transformação para o caliper, que exigiu revisitar álgebra linear aplicada — um domínio fora da zona de conforto inicial, mas essencial para ferramentas de medição precisas em ambiente clínico.

Consideração Final

Em um contexto de imagens médicas como DICOM, o pipeline de processamento implementado — LUT de 256 entradas, acesso direto a `scanLine`, interpolação adaptativa — pode ser expandido para suportar imagens de 16 bits e metadados DICOM reais, garantindo fidelidade diagnóstica sem comprometer a performance.